

第8讲

Attention 到底是什么？

从相关性、Softmax 到 Query / Key / Value

本讲目标

学完后，你应该能用自己的话解释：Attention 为什么需要“相关性分数”；Softmax 在做什么；Q、K、V 各自扮演什么角色；为什么 Q/K/V 都来自可学习线性投影；完整公式每一项是什么意思；以及自回归 LLM 为什么必须加 Causal Mask。

先记住一句话

一句话心智模型

Attention = 用 Query 去和所有 Key 比较，得到权重，再按照这些权重读取对应的 Value。

也就是：“我现在需要什么？” → “谁最相关？” → “从相关位置读取多少内容？”

Attention 最小闭环：先比较，再分权重，最后读取并汇总



真正的 Self-Attention 会让每个 token 都执行类似过程

1. 先看公式：Attention 解决的到底是什么问题？

上一讲我们看到，RNN/LSTM 主要靠状态一步步传递信息。Attention 改变了思路：当前计算可以直接访问多个位置，再决定哪些位置更重要。

例子

句子：“小明把书借给小王，因为他已经看完了。”

当模型处理“他”时，可能需要比较“小明”“书”“小王”等位置，然后根据上下文形成新的表示。关键不是“只选一个词”，而是给不同位置不同权重。

所以 Attention 最小可以拆成三步：

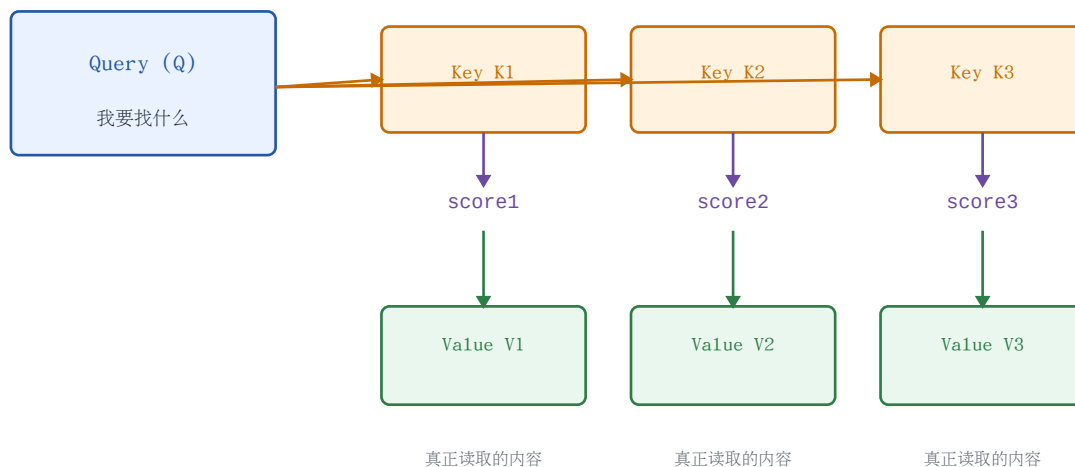
- 比较：当前位置和其他位置有多相关？
- 归一化：把相关性分数变成可比较的权重。
- 读取：按权重把各位置的信息加权汇总。

重要

Attention 不是“从一句话里挑一个词”。它通常会把很多位置都纳入计算，只是权重不同。

2. Query、Key、Value：三个名字到底在说什么？

类比：Q 像搜索请求，K 像索引标签，V 像标签背后的正文内容



这是理解 Q/K/V 最好用的第一层直觉：

角色	直觉	在 Attention 中做什么
Query (Q)	“我现在想找什么？”	拿它去和所有 Key 比较
Key (K)	“我有什么可被匹配的标签？”	决定自己和 Query 有多匹配
Value (V)	“如果你关注我，真正应该读取什么？”	被权重加权后汇总到输出

图书馆比喻

Query 像你输入的检索词；Key 像每本书的主题索引；Value 像书里的真正内容。你先靠 Query 和 Key 找相关书，再去读取对应 Value。

3. 为什么 Key 和 Value 要分开？

初学者最常问：“既然都是同一个 token，为什么不能拿同一个向量既匹配又读取？”

因为“用于判断相关性的信息”和“真正要传递的内容”可以是不同的。

例子

你在数据库里查论文时：

- 关键词、标题、标签适合做匹配；
- 论文正文才是你真正要读取的内容。

如果把“索引字段”和“正文内容”完全绑在一起，系统的表达能力会更受限制。

神经网络让 Key 空间专门学习“怎样被查询”，Value

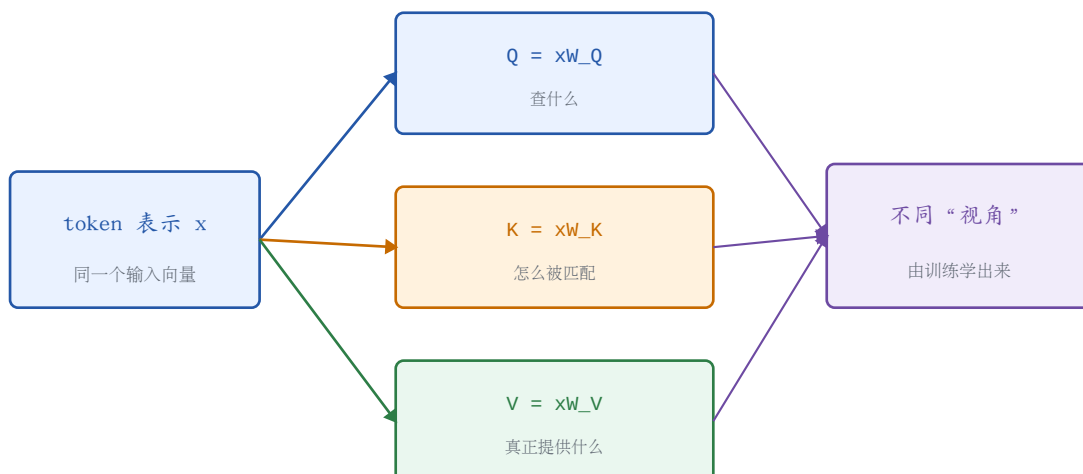
空间专门学习“被选中后提供什么信息”。这给模型更大的自由度。

不要误解

这只是功能上的类比。真实模型里的 K 和 V 都是连续向量，并不是人手写的标签或正文。

4. Q、K、V 从哪里来？不是凭空出现

同一个 token 表示，通过三个不同的可学习矩阵变成 Q、K、V



假设某个 token 当前表示为向量 x 。模型会用三个不同的可学习矩阵做线性变换：

$$Q = xW_Q \quad K = xW_K \quad V = xW_V$$

这里的 W_Q 、 W_K 、 W_V 就像上一阶段学过的神经网络权重。训练时，它们会通过梯度下降被不断调整。

矩阵	学习目标的直觉
W_Q	把当前表示投影成“查询需求”的视角
W_K	把当前表示投影成“可匹配标签”的视角
W_V	把当前表示投影成“可被读取内容”的视角

关键

同一个 token 并不是只有一个“固定意义”。在同一层 Attention 中，它会被投影成 Q、K、V 三种不同角色。

5. 相关性怎么计算？最经典的是点积

最常见的 Scaled Dot-Product Attention 先用 Query 和 Key 做点积：

```
score_i = Q dot K_i
```

如果 Q 和 K 在某些维度方向上比较一致，点积通常更大；如果方向相反，可能得到较小甚至负的分。

二维小例子

$Q = [1, 0]$

$K1 = [2, 0] \rightarrow \text{点积} = 2$

$K2 = [1, 1] \rightarrow \text{点积} = 1$

$K3 = [0, 1] \rightarrow \text{点积} = 0$

那么 K1 在当前 Q 下最匹配。

注意：点积并不是唯一可能的相似度函数。但 Transformer 经典实现选择了高效的矩阵乘法形式，因此非常适合 GPU。

6. 为什么还要除以 $\sqrt{d_k}$?

完整公式不是 QK^T ，而是：

```
scores = (Q K^T) / sqrt(d_k)
```

这里 d_k 是 Key/Query 的维度。维度变大后，点积的绝对值往往也会变大。如果把很大的分数直接送入 Softmax，Softmax 可能变得极端尖锐，梯度也更不稳定。

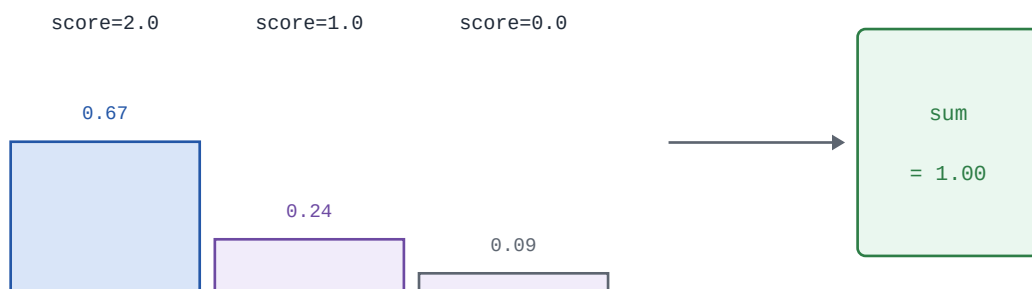
直觉

除以 $\sqrt{d_k}$ 相当于给分数做“温度校准”，避免维度一大，分数就失控。它不是为了改变谁更相关的基本排序，而是为了让数值尺度更稳定。

你现在不需要推导方差公式。只要知道：Scale 的作用是数值稳定和训练稳定。

7. Softmax：把分数变成“读取比例”

Softmax：把任意实数分数变成“全为正、总和为 1”的权重



分数差距越大，Softmax 后的权重通常越“尖锐”

Softmax 接受一组任意实数分数，然后输出一组权重：

```
weight_i = exp(score_i) / sum_j exp(score_j)
```

它有两个非常重要的性质：

- 所有权重都大于 0；
- 所有权重加起来等于 1。

所以我们可以把它直觉理解成“当前 Query 准备把多少读取预算分给每个位置”。

但别把它当概率真理

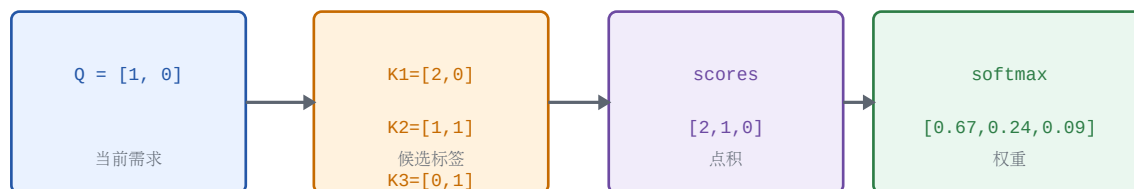
数学上这些值像概率分布，但它们首先是神经网络内部的归一化权重，不代表“模型相信某事实的真实概率”。

8. 最后一步：按权重读取 Value

得到权重以后，模型不会把 Key 加权求和，而是去加权 Value：

```
output = sum_i weight_i * V_i
```

一个 Query 的完整数值例子（教学简化）



■ $V1=[10, 0]$, $V2=[0, 10]$, $V3=[5, 5]$

$output = 0.67*V1 + 0.24*V2 + 0.09*V3 = [7.15, 2.85]$

这一步非常关键：Key 负责“被匹配”，Value 负责“被读取”。

一句话

Q 和 K 决定“看谁”；V 决定“看到了什么”。

9. 把三步合起来：完整 Attention 公式

现在你已经可以真正读懂经典公式：

Attention(Q,K,V) = softmax(Q K^T / sqrt(d_k)) V

不要一次把它当成一大坨数学。把它从左到右拆开：

步骤	公式部分	在做什么
1	$Q K^T$	每个 Query 与所有 Key 做点积，得到相关性分数
2	$/ \sqrt{d_k}$	控制分数尺度，提升数值稳定性
3	$\text{softmax}(\dots)$	把分数变成和为 1 的权重
4	$\dots V$	按权重读取并汇总 Value

你真正要会的

看到这个公式时，脑子里应该出现：匹配 -> 分权 -> 读取。公式只是这三个动作的矩阵写法。

10. 为什么公式里出现 K^T ?

如果 Q 是“所有 token 的 Query 矩阵”，K 是“所有 token 的 Key 矩阵”，我们希望一次性计算每个 Query 对每个 Key 的点积。

假设:

```
Q shape = [n, d_k] K shape = [n, d_k] K^T shape = [d_k, n] Q K^T shape = [n, n]
```

于是 n 个 token 一次矩阵乘法就得到一个 $n \times n$ 的分数矩阵。
Key

Attention Matrix: 每一行表示“某个机器喜欢”Query 看答Key 的权重”

我	0.55	0.20	0.15	0.10
喜欢	0.25	0.50	0.15	0.10
Query				
机器	0.10	0.20	0.55	0.15
学习	0.10	0.15	0.25	0.50

系统视角

这正是 Transformer 非常适合现代 GPU 的原因之一：很多本来像“逐个比较”的操作，可以重写成大矩阵乘法并行执行。

11. Attention Matrix 怎么读？

对一个 n -token 序列，Attention 权重可以看成是一个 $n \times n$ 矩阵。通常：

- 第 i 行：第 i 个 Query 对所有 Key 的权重；
- 第 j 列：不同 Query 对第 j 个位置分了多少注意力。

每一行经过 Softmax 后通常和为 1。

重要提醒

Attention Matrix 看起来很像“模型的关系图”，但不要直接把一张 Attention 热图解释成人类可读的因果机制。注意力权重只是网络中的一个中间变量。

后面 Multi-Head Attention 中，同一层还有多个不同的 Attention Matrix；更高层又会继续变换，因此最终行为并不能由单张图完全解释。

12. Self-Attention: Q、K、V 都来自同一序列

Self-Attention 的“Self”不是说一个 token 只看自己，而是说：Q、K、V 的源头来自同一组序列表示。

例如句子有 4 个 token:

X = [x1, x2, x3, x4] Q = X W_Q K = X W_K V = X W_V

然后每个 Query 都可以和同一序列的所有 Key 比较，再读取同一序列各位置的 Value。

类型	Q 来自	K/V 来自
Self-Attention	当前序列	当前序列
Cross-Attention	通常来自 Decoder 当前表示	通常来自 Encoder / 外部序列表示

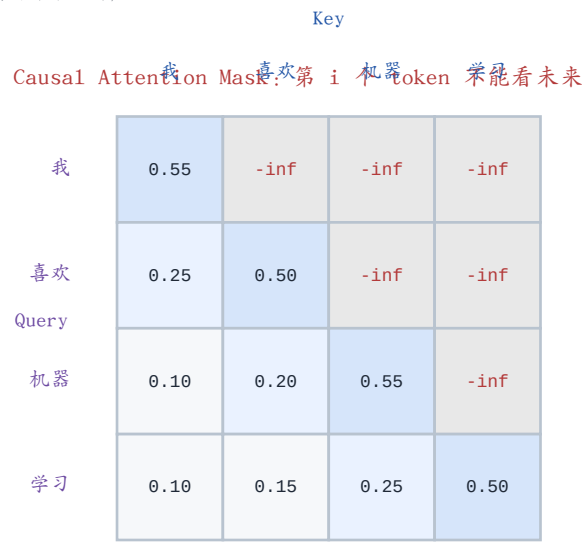
GPT 重点

Decoder-only GPT 的主体使用的是带 Causal Mask 的 Self-Attention。

13. 关键问题：GPT 为什么不能“偷看未来”？

GPT 的训练目标是根据前文预测下一个 token。如果训练第 3 个位置时允许它直接看到第 4、5、6 个真实 token，那任务就作弊了。

因此需要 Causal Mask（因果掩码）。



训练自回归 LLM 时，未来位置先被设成 -inf，Softmax 后其权重变成 0

通常做法是：在 Softmax 之前，把“未来位置”的分数加上一个非常大的负数，概念上可看作 -infinity。于是：

`softmax(-infinity) -> 0`

结果

第 i 个 token 只能关注自己和左边已经出现的位置，不能关注右边未来 token。这保证了自回归生成的因果方向。

14. Causal Mask 和 Padding Mask 不是一回事

Mask	目的	例子
Causal Mask	禁止看未来 token	GPT 预测下一个 token 时不能看到答案
Padding Mask	忽略为了对齐长度而补的无意义 token	一个 batch 中短句被补到和长句一样长

二者都通过修改 Attention 分数，让某些位置在 Softmax 后得到 0 权重，但语义完全不同。

实验阅读提示

以后看论文或代码时遇到 attention_mask，不要自动以为就是 Causal Mask。一定看它是在屏蔽未来、Padding，还是其他自定义结构。

15. Attention 输出后，token 发生了什么变化？

在进入 Attention 之前，每个位置都有一个向量表示。Attention 之后，每个位置得到一个新的输出向量，这个向量已经混入了其他位置的信息。

可以把它理解为：

原始 x_i ：更多是“这个 token 当前自身的表示” Attention output_i：加入“根据当前 Query 从上下文读取来的信息”

上下文化表示

因此同一个词在不同句子里，经过 Attention 后可以形成不同表示。比如 “bank” 在 “bank account” 和 “river bank” 中，会从不同上下文位置读取不同信息。

后面的 Transformer 层还会继续重复这种“读取上下文 -> 变换表示”的过程。

16. 为什么 Q/K/V 需要“可学习”？

如果我们手工规定“名词看名词”“代词看人物”，系统只能处理设计者预先想到的关系。Transformer 选择让 W_Q 、 W_K 、 W_V 通过训练自己学。

当预测错误产生 Loss 时，梯度会一路传回这些矩阵，于是模型逐渐学到：

- 什么样的 Query 应该与什么样的 Key 匹配；
- 哪些内容应该被编码进 Value；
- 不同层中哪些关系值得被强调。

底层连接

你现在可以把第5讲的训练闭环接到 Attention：

预测错误 $\rightarrow$ Loss $\rightarrow$ Backprop $\rightarrow$ 更新 $W_Q/W_K/W_V \rightarrow$ 下一次 Attention 的匹配与读取方式改变。

17. 一个完整 Self-Attention 的计算流程

假设输入矩阵 X 包含 n 个 token 的当前表示:

```
1.  $Q = X W_Q$  2.  $K = X W_K$  3.  $V = X W_V$  4.  $S = Q K^T / \sqrt{d_k}$  5. 如果是 GPT:  $S = S + \text{causal\_mask}$  6.  $A = \text{softmax}(S)$  7.  $O = A V$ 
```

其中:

符号	含义
X	本层输入的 token 表示
$Q/K/V$	三种学习出来的投影视角
S	未归一化相关性分数矩阵
A	Attention 权重矩阵
O	融合上下文后的输出表示

最低闭环能力

如果你能把这 7 行逐行解释清楚，就已经真正跨过了理解 Transformer 的第一道核心门槛。

18. Attention 的计算成本从哪里来？

最关键的一步是 QK^T 。序列长度 n 时，会形成 $n \times n$ 的位置关系矩阵。

序列长度 n	Attention 位置对数量 n
1,000	1,000,000
8,000	64,000,000
32,000	1,024,000,000
128,000	16,384,000,000

因此长上下文会带来计算和显存压力。这就是为什么后续会出现 FlashAttention、稀疏 Attention、线性 Attention、分块策略等大量系统与算法工作。

注意

这里的 n 是标准全注意力位置关系规模的直觉。实际推理成本还涉及 Prefill、Decode、KV Cache、批大小、实现优化和硬件。

19. Attention 权重能不能直接解释模型？

这是科研中很容易踩坑的问题。一个 Attention Head 的高权重只能说明：在这一步加权读取里，这些 Value 被分配了较高系数。

它不自动等于：

- 这个 token 是模型做出最终答案的唯一原因；
- 这个 token 在所有层、所有 Head 都重要；
- 把高权重 token 删除后，模型一定会按你想象的方式改变；
- Attention 热图就是完整的“思维过程”。

科研判断

如果论文要用 Attention

权重做机制解释，通常还需要干预实验、消融、因果测试或其他证据，而不是只展示热图。

20. Attention 和 Agent Systems 有什么直接关系？

Agent 的 Prompt、Memory、工具结果、多 Agent 消息，最终都会成为上下文 token。模型能否把正确的信息“读出来”，底层就和 Attention 相关。

- 长任务遗忘规则：规则虽然还在 Context 中，但不代表模型一定有效利用。
- 工具结果过长：大量低价值 token 会增加 Attention 计算和信息竞争。
- Memory 检索：外部检索决定“把什么放进上下文”，Attention 决定“模型怎样利用已经放进来的内容”。
- 多 Agent 通信：通信越长，token 成本越高，模型需要在更多位置之间分配读取权重。
- 可靠性实验：研究“重要约束放在不同位置是否会被稳定遵循”，本质上部分涉及上下文利用机制。

科研边界

但不要把所有 Agent 失败都归因于 Attention。规划、采样、工具接口、Memory 策略、Prompt、模型能力和环境反馈都可能造成失败。

21. 常见误区

误区1: Q、K、V 是三个不同 token

不是。每个 token 都会产生自己的 Q、K、V。

误区2: Q/K/V 是人工写好的语义标签

不是。它们是可学习线性投影后的连续向量。

误区3: Key 就是要读取的内容

Key 主要用于匹配；真正被加权汇总的是 Value。

误区4: Softmax 最大的位置就是唯一答案

Attention 通常是软加权，很多位置都可以参与。

误区5: Attention 权重就是模型思维过程

过度解释。它只是网络内部的一个中间加权机制。

误区6: Self-Attention 就是 token 只关注自己

“Self” 指 Q/K/V 来自同一序列，不是只能看自身。

误区7: GPT 训练时可以看完整句子，所以会偷看答案

Causal Mask 会屏蔽未来位置。

误区8: Attention 只是数学技巧，与系统无关

QK^T 的 n^2 规模直接影响长上下文算力、显存和系统设计。

22. 小实验：把 Q/K/V 的每一步打印出来

材料包里的 `04_attention_qkv_demo.py` 只使用 Python 标准库，不依赖 PyTorch。它会完成两个实验：

- 实验 A：一个 Query 对三个 Key 计算点积 $\rightarrow$ Softmax $\rightarrow$ 加权 Value；
- 实验 B：四个 token 构造一个简化 Attention Matrix，并演示 Causal Mask 后未来位置权重如何变成 0。

你应该重点观察输出中四件事：

- scores 可以是任意实数；
- weights 全为正且总和约等于 1；
- 最高 score 通常对应最高 weight；
- 被 Mask 的未来位置最终 weight 等于 0。

实验目的

不是训练模型，而是把 Attention 从“公式”变成你亲眼看到的数值流水线。

23. 本讲总结：把 Attention 压缩成 6 句话

Attention 最小闭环：先比较，再分权重，最后读取并汇总



真正的 Self-Attention 会让每个 token 都执行类似过程

- 每个 token 的当前表示通过 W_Q 、 W_K 、 W_V 变成 Q 、 K 、 V 。
- Q 和所有 K 做点积，得到相关性分数。
- 分数除以 $\sqrt{d_k}$ ，控制数值尺度。
- Softmax 把分数变成和为 1 的权重。
- 这些权重去加权 V ，得到融合上下文后的输出。
- GPT 再加 Causal Mask，保证当前位置不能读取未来 token。

完整公式

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$$

记忆口令：Q 找 K，权重读 V。

24. 术语表

术语	英文	零基础解释
查询向量	Query / Q	当前位置想找什么
键向量	Key / K	当前位置怎样被别人匹配
值向量	Value / V	被关注后真正贡献的内容
点积	Dot Product	一种高效的向量匹配分数
缩放	Scaling	除以 $\sqrt{d_k}$ 控制分数尺度
Softmax	Softmax	把分数变成正数且和为 1 的权重
注意力权重	Attention Weight	当前 Query 分给每个位置的读取比例
注意力矩阵	Attention Matrix	所有 Query 对所有 Key 的权重表
自注意力	Self-Attention	Q/K/V 来自同一序列
交叉注意力	Cross-Attention	Q 和 K/V 来自不同序列/来源
因果掩码	Causal Mask	禁止自回归模型看到未来 token
填充掩码	Padding Mask	忽略对齐长度时补出的无意义位置
上下文化表示	Contextualized Representation	融合上下文后得到的新 token 表示

25. 练习题（先自己做）

- 1. 用一句话解释 Attention 的三步核心流程。
- 2. Query、Key、Value 分别是什么角色？
- 3. 为什么 Key 和 Value 不一定要是同一个表示？
- 4. Q、K、V 是从哪里来的？训练时哪些参数会被更新？
- 5. 点积 $Q \text{ dot } K$ 在 Attention 中表示什么？
- 6. 为什么要除以 $\sqrt{d_k}$ ？
- 7. Softmax 输出有什么两个重要性质？
- 8. 为什么最终加权的是 V，而不是 K？
- 9. 请逐项解释 $\text{Attention}(Q,K,V) = \text{softmax}(QK^T/\sqrt{d_k})V$ 。
- 10. 如果 $Q \text{ shape}=[n,d_k]$ ， $K \text{ shape}=[n,d_k]$ ，为什么 QK^T 是 $[n,n]$ ？
- 11. Self-Attention 中“Self”到底指什么？
- 12. GPT 为什么必须使用 Causal Mask？
- 13. Causal Mask 和 Padding Mask 有什么不同？
- 14. 为什么 Attention 输出可以说是“上下文表示”？
- 15. Attention 权重高，是否等于这个 token 是最终答案的因果原因？为什么？
- 16. 标准全注意力为什么会带来 n^2 级别的长上下文压力？
- 17. 结合 Agent Systems：外部 Memory 检索和模型内部 Attention 分别解决什么问题？
- 18. 请画出完整流程： $X \rightarrow Q/K/V \rightarrow \text{scores} \rightarrow \text{mask} \rightarrow \text{softmax} \rightarrow \text{weighted } V \rightarrow \text{output}$ 。

26. 练习题答案与自检

第1题

先计算当前 Query 与各 Key 的相关性，再用 Softmax 变成权重，最后按权重加权对应 Value。

第2题

Q 表示“我想找什么”；K 表示“我怎样被匹配”；V 表示“如果你关注我，我真正提供什么内容”。

第3题

匹配所需特征和真正要传递的信息可以不同，分开后模型表达能力更强。

第4题

来自输入表示 X 乘以可学习矩阵 W_Q 、 W_K 、 W_V ；训练时这些矩阵会通过反向传播更新。

第5题

表示当前 Query 与某个 Key 的匹配程度/相关性分数。

第6题

高维点积绝对值容易变大，缩放可避免 Softmax 过度饱和并改善数值和训练稳定性。

第7题

所有输出都为正；所有权重总和为 1。

第8题

K 用来决定是否相关；V 才承载要被读取并传递到输出的内容。

第9题

QK^T 计算所有 Query-Key 分数；除 $\sqrt{d_k}$ 缩放；Softmax 归一化；最后乘 V 做加权读取。

第10题

K^T 是 $[d_k, n]$ ，矩阵乘法 $[n, d_k] \times [d_k, n]$ 得到 $[n, n]$ 。

第11题

Q、K、V 的源头来自同一序列的表示，而不是说 token 只能看自己。

第12题

因为 GPT 要学习根据过去预测未来；若看到未来真实 token 就会泄露答案。

第13题

Causal Mask 屏蔽未来位置；Padding Mask 屏蔽为了批处理对齐而补的无意义位置。

第14题

因为输出不只包含当前位置自身信息，还融合了按权重从上下文其他位置读取的信息。

第15题

不等于。Attention 权重只是一个中间计算量，最终行为还经过多个 Head、层、残差和 FFN；机制解释通常需要干预证据。

第16题

n 个 Query 都可能和 n 个 Key 比较，因此关系矩阵规模是 $n \times n$ 。

第17题

检索负责决定“把哪些外部信息放进上下文”；Attention 负责模型内部“怎样利用已经进入上下文的信息”。

第18题

自检图应包含： $X \rightarrow$ 线性投影得到 $Q/K/V \rightarrow QK^T/\text{sqrt}(d_k) \rightarrow$ 加 Mask $\rightarrow$ Softmax $\rightarrow$ 权重乘 $V \rightarrow$ 输出。

27. 原始论文位置与下一讲

本讲核心公式来自 Transformer 奠基论文中的 Scaled Dot-Product Attention。现阶段你已经具备读懂这部分核心公式的最低基础。

工作	你现在应该能读懂什么
Attention Is All You Need (2017)	Scaled Dot-Product Attention 公式、Q/K/V、Mask 的基本意义
后续 Multi-Head Attention	为什么不只做一套 Q/K/V，而是同时做多个“头”
现代 Decoder-only LLM	带 Causal Mask 的 Self-Attention 是主干之一

下一讲

第9讲进入 Multi-Head Attention: 为什么一个注意力头不够？我们会解释 Head 是什么、不同 Head 如何并行、Concat + W_O 在做什么，以及“多个视角”为什么有用。